

Assignment 2 – Reliable and causal-total group communication

Develop group communication support in the spirit of reliable causal-total multicast. Your implementation must provide a suitable interface for the application, such as the following (indicative) API:

<code>int grp_init(char *gms_addr, int gms_port)</code>	Initialization with GMS info
<code>int grp_join(char *grpname, char *myid);</code>	Adds the calling process in a group
<code>int grp_leave(int g);</code>	Removes the calling process from a group
<code>int grp_send(int g, void *msg, int len);</code>	Send group message
<code>int grp_recv(int g, void *msg, int *len, int block);</code>	Receive next group message

Group management. Group membership is managed via a separate group management service (GMS). The `grp_join()` primitive communicates with the GMS to send the process identifier and contact information and to receive information about the members of the group. It returns an integer (in the spirit of a socket/file descriptor) through which the application can subsequently send/receive group messages in the context of that group. The `grp_leave()` primitive communicates with the GMS to remove the process from the group. Each time the group membership changes, the GMS notifies all member processes accordingly. Communication with the GMS is over TCP/IP. For simplicity, the contact information of the GMS is passed as a parameter in `grp_init()`. Note that the GMS should not be involved in the actual messaging between members.

Application-level group view. Think how the application learns about membership changes. Adapt/augment the above API or introduce specially typed messages. For simplicity, assume that the addition/removal of processes in/from groups occurs only at “safe” periods where there are no messages in transit. However, process failures are an exception (see below).

Reliable causal-total multicast. Implement message propagation within the group using UDP/IP unicasts. You may assume that group messages fit into UDP datagrams. All group messages must be delivered to the application at all member processes in causal-total order, despite any message delays, loss, duplication or reordering that may occur by the network. Use whichever protocol you wish (coordinated or symmetrical – see lectures). In any case, you must properly handle group membership changes, ensuring that message propagation/delivery continues smoothly. Same applies to process failures (see below).

Measurements. Measure the throughput of your implementation as a function of group size N (e.g., $N=2,3,4,5$), for the case where (i) only one process is sending messages to the group, and (ii) all processes are sending messages concurrently. Also, measure the communication overhead of your implementation by counting the average number of protocol-level messages sent for each application-level message. Confirm what you measure is consistent with your own “theoretical” estimate.

Demonstration application. Demonstrate your implementation using a simple group chat application. Think about how to simulate message loss in a practical way but also how to automate correctness verification for a large number of messages.

Process failures. Extend your implementation to handle process failures. Assign the responsibility of liveness checking to the GMS (which can be assumed to be reliable). If the GMS detects a process failure, it must notify all (remaining) group members. Failures may occur anytime, also during group message propagation, which must be extended to work in a robust way. Finally, the application must be informed about failures in the same way this is done for processes that leave the group.

Optional (once everything else works robustly)

Use UDP/IP multicasting for more efficient message propagation in the group, instead of sending every message multiple times separately to each member via a UDP/IP unicast. Repeat the same measurements as above.

Further clarifications

No assumptions can be made about the contents of application messages or when the application decides to send/receive messages. The group communication layer must run decoupled from the application thread. The same application process can participate in different groups by invoking `join()` multiple times for the respective group names. You may use any programming language you wish, provided it supports true threads/concurrency.

Submission deadline (see instructions in eclass): Saturday 28 March 2026, 22:00
--